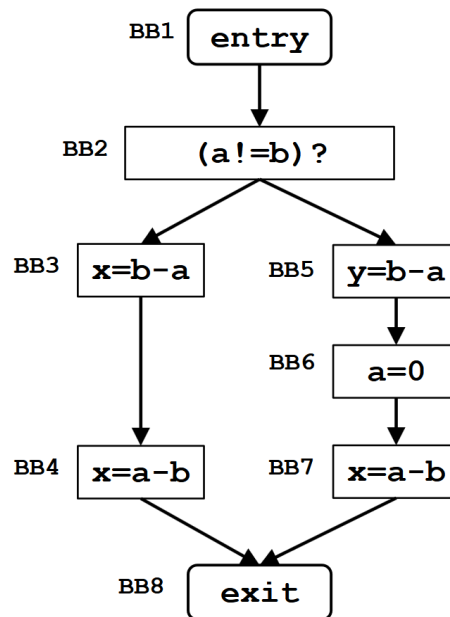


Secondo Assignment - Dataflow Analysis

1. Very Busy Expressions

Un'espressione $a \text{ op } b$ è **very busy** in un punto p se, indipendentemente dal percorso preso da p , l'espressione viene usata prima che uno dei suoi operandi venga definito.



Dominio e Direzione

Il dominio di questa analisi è l'**insieme delle espressioni**.

La direzione dell'analisi è **backward**, dato che per sapere se un'espressione è very busy in un determinato blocco si deve guardare cosa succede nei blocchi successivi.

Insiemi *GEN* e *KILL*

Data un'espressione $a \text{ op } b$, $a \text{ op } b \in \text{GEN}[B]$ se l'espressione appare nel blocco B e non sono presenti assegnazioni ad a o b all'interno del blocco B che precedono la prima occorrenza dell'espressione.

Data un'espressione $a \text{ op } b$, $a \text{ op } b \in \text{KILL}[B]$ se esiste un'assegnazione ad a o b all'interno del blocco B .

Equazioni *IN* e *OUT*

Poiché questo è un problema bit-vector standard con direzione backward, le equazioni *IN* e *OUT* sono le seguenti:

$$IN[B] = f_B(OUT[B])$$

$$OUT[B] = \bigwedge_{s \in \text{succ}(B)} (IN[s])$$

Transfer function

La funzione di trasferimento, come in tutti gli altri casi di bit-vector, è la seguente:

$$f_B(x) = GEN[B] \cup (x - KILL[B])$$

Meet Operation

Per determinare l'insieme di espressioni very busy si deve prestare attenzione a ogni percorso che consente di arrivare in un determinato punto del CFG, motivo per cui l'operatore di meet deve essere l'**intersezione**.

Boundary Condition

Poiché al termine del flusso di esecuzione (*EXIT*) non sono presenti valutazioni successive da soddisfare, l'insieme delle espressioni deve essere vuoto. Si pone quindi $IN[EXIT] = \emptyset$ come boundary condition.

Initial interior points

L'uso dell'intersezione come operatore di meet impone un'inizializzazione conservativa per i nodi interni. Si fissa pertanto $IN[B] = u$ al passo base dell'iterazione, dove u rappresenta la totalità delle espressioni definite nel programma.

Framework di Dataflow Analysis

	Very Busy Expressions
Domain	Espressioni
Direction	Backward: $IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{s \in succ(B)} (IN[s])$
Transfer function	$f_B(x) = GEN[B] \cup (x - KILL[B])$
Meet Operation ($\wedge$)	Intersezione ($\cap$)
Boundary Condition	$IN[EXIT] = \emptyset$
Initial interior points	$IN[B] = u$

Si ricavano gli insiemi $GEN[B]$ e $KILL[B]$ per ogni Basic Block B .

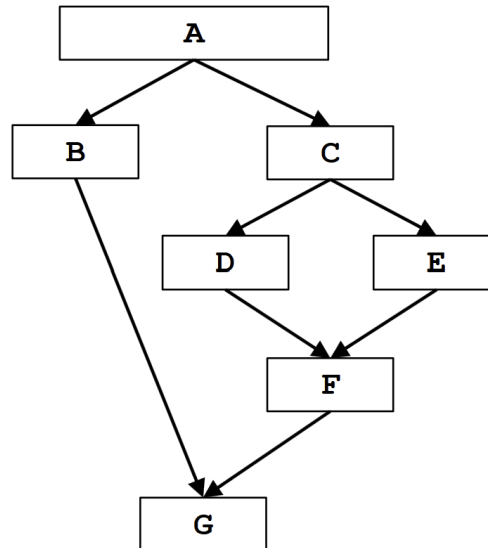
Basic Block	$GEN[B]$	$KILL[B]$
BB_1	$\emptyset$	$\emptyset$
BB_2	$\emptyset$	$\emptyset$
BB_3	$\{b - a\}$	$\emptyset$
BB_4	$\{a - b\}$	$\emptyset$
BB_5	$\{b - a\}$	$\emptyset$
BB_6	$\emptyset$	$\{b - a, a - b\}$
BB_7	$\{a - b\}$	$\emptyset$
BB_8	$\emptyset$	$\emptyset$

Sulla base del CFG riportato e degli insiemi GEN e $KILL$, si esegue l'algoritmo per il calcolo degli insiemi $IN[B]$ e $OUT[B]$ per ogni Basic Block B .

Basic Block	$IN[B]$	$OUT[B]$
BB_1	$\{b - a\}$	$\{b - a\}$
BB_2	$\{b - a\}$	$\{b - a\}$
BB_3	$\{b - a, a - b\}$	$\{a - b\}$
BB_4	$\{a - b\}$	$\emptyset$
BB_5	$\{b - a\}$	$\emptyset$
BB_6	$\emptyset$	$\{a - b\}$
BB_7	$\{a - b\}$	$\emptyset$
BB_8	$\emptyset$	$\emptyset$

2. Dominator Analysis

In un CFG un nodo X **domina** un nodo Y se X appare in ogni percorso del grafo che porta dall'ingresso del programma (blocco *ENTRY*) al blocco Y .



Dominio e Direzione

Il dominio di questa analisi è l'**insieme dei Basic Blocks**.

La direzione dell'analisi è **forward**, dato che l'insieme dei dominatori di un blocco B è legato ai suoi predecessori, cioè i blocchi da attraversare prima di raggiungere B .

Insiemi *GEN* e *KILL*

$GEN[B] = \{B\}$. Per costruzione dell'analisi, ogni Basic Block domina sé stesso. Di conseguenza, l'attraversamento di un nodo comporta l'immissione del nodo stesso nell'insieme dei dominatori.

$KILL[B] = \emptyset$. Poiché l'acquisizione di un dominatore lungo un percorso non può mai essere annullata dai blocchi successivi, l'insieme delle proprietà soppresse risulta essere vuoto per la totalità dei nodi.

Equazioni *IN* e *OUT*

Poiché questo è un problema bit-vector standard con direzione forward, le equazioni *IN* e *OUT* sono le seguenti:

$$IN[B] = \bigwedge_{p \in pred(B)} (OUT[p])$$
$$OUT[B] = f_B(IN[B])$$

Transfer function

La funzione di trasferimento, come in tutti gli altri casi di bit-vector, è la seguente:

$$f_B(x) = GEN[B] \cup (x - KILL[B]) = GEN[B] \cup x$$

Essendo $KILL[B]$ un insieme sempre vuoto.

Meet Operation

La definizione stessa di dominanza richiede che una proprietà sia verificata lungo l'intersezione dei cammini convergenti in un nodo. Di conseguenza, l'operatore di meet appropriato per mantenere sole le proprietà comuni è l'**intersezione**.

Boundary Condition

Dal momento che il blocco *ENTRY* non può essere dominato da nessun altro blocco, si pone $IN[ENTRY] = \emptyset$ come boundary condition.

Initial interior points

Analogamente al problema precedente, la natura intersecante dell'operatore di meet richiede un'inizializzazione limite per i nodi interni. Si imposta quindi $OUT[B] = u$ al passo iniziale, dove u rappresenta la totalità dei Basic Block definiti nel programma.

Framework di Dataflow Analysis

	Dominator Analysis
Domain	Basic Block
Direction	Forward: $IN[B] = \bigwedge_{p \in pred(B)} (OUT[p])$ $OUT[B] = f_B(IN[B])$
Transfer function	$f_B(x) = GEN[B] \cup x$
Meet Operation ($\wedge$)	Intersezione ($\cap$)
Boundary Condition	$IN[ENTRY] = \emptyset$
Initial interior points	$OUT[B] = u$

Si ricavano gli insiemi $GEN[B]$ e $KILL[B]$ per ogni Basic Block B .

Basic Block	$GEN[B]$	$KILL[B]$
BB_A	$\{A\}$	$\emptyset$
BB_B	$\{B\}$	$\emptyset$
BB_C	$\{C\}$	$\emptyset$
BB_D	$\{D\}$	$\emptyset$
BB_E	$\{E\}$	$\emptyset$
BB_F	$\{F\}$	$\emptyset$
BB_G	$\{G\}$	$\emptyset$

Sulla base del CFG riportato e degli insiemi GEN e $KILL$, si esegue l'algoritmo per il calcolo degli insiemi $IN[B]$ e $OUT[B]$ per ogni Basic Block B .

Basic Block	$IN[B]$	$OUT[B]$
BB_A	$\emptyset$	$\{A\}$
BB_B	$\{A\}$	$\{A, B\}$
BB_C	$\{A\}$	$\{A, C\}$
BB_D	$\{A, C\}$	$\{A, C, D\}$
BB_E	$\{A, C\}$	$\{A, C, E\}$
BB_F	$\{A, C\}$	$\{A, C, F\}$
BB_G	$\{A\}$	$\{A, G\}$

3. Constant Propagation

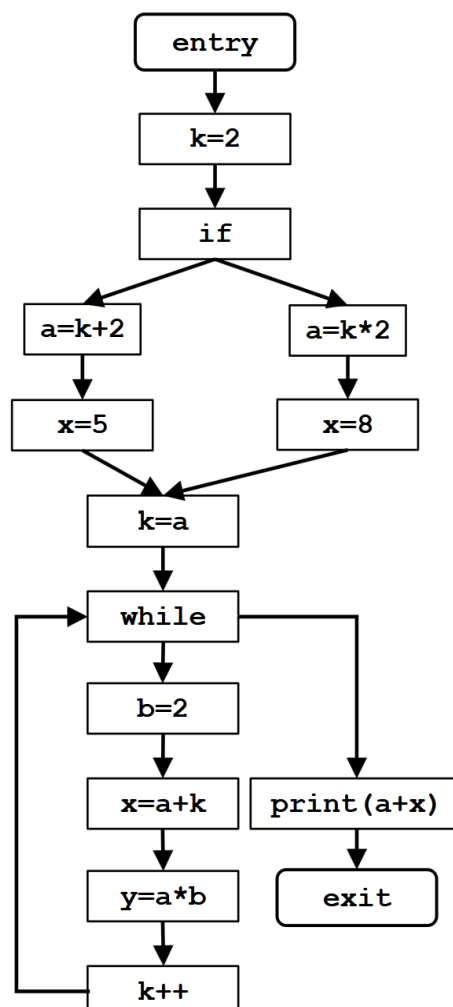
L'obiettivo della Constant Propagation è quello di determinare **in quali punti del programma le variabili hanno valore costante**.

L'informazione da calcolare per ogni nodo del CFG è un insieme di coppie del tipo $\langle \text{variabile}, \text{valore costante} \rangle$.

Se si ha la coppia $\langle x, c \rangle$ al nodo n , significa che x è garantito avere valore c ogni volta che n viene raggiunto durante l'esecuzione del programma.

L'analisi di Constant Propagation riesce a determinare il valore costante di espressioni binarie in cui uno o entrambi gli operandi siano delle variabili il cui valore costante sia noto:

```
w = 5
x = 12
y = x - 2      → y = 10
z = w + x      → z = 17
```



Limiti dell'approccio bit-vector

La risoluzione del problema di Constant Propagation non è compatibile con i framework DFA basati su bit-vector a causa di due limitazioni strutturali fondamentali:

1. Dominio e cardinalità infinita

L'informazione propagata nel CFG non è esprimibile come una proprietà booleana di appartenenza, ma è una mappatura del tipo $\langle \text{variabile}, \text{valore costante} \rangle$. Poiché le variabili possono assumere un numero arbitrariamente grande di valori numerici, il dominio non è finito.

Risulta pertanto matematicamente e spazialmente impossibile pre-allocare un vettore di bit in grado di rappresentare lo stato di ogni possibile costante per ciascuna variabile.

2. Dinamicità della Funzione di Trasferimento

Nel framework standard, l'impatto logico di una singola istruzione è statico, rendendo gli insiemi generati e soppressi pre-calcolabili. Al contrario, nella Constant Propagation, la validità di un'espressione dipende in modo iterativo dallo stato di esecuzione computato ai passi precedenti. La funzione di trasferimento deve quindi operare dinamicamente per gestire questi insiemi non predicibili a priori.

Conclusione

A causa della natura del problema, l'approccio DFA standard non è sufficiente. È necessario appoggiarsi a una struttura dati più complessa che permetta di mappare dinamicamente le variabili in uno spazio di valori teoricamente infinito, tracciando esplicitamente i punti del programma in cui le costanti sono garantite durante l'esecuzione.